Answer Key — Competitive Exam Practice 3: System Calls

GATE (Computer Science) Pattern

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

October 17, 2026

Concepts That Click

Q1 (B). A library call is a convenience layer, not a trap. `printf` formats into a user-space buffer and traps only when the buffer is flushed, and a buffered `fread` may return data without any system call. (A) is false because one library call may issue zero, one, or many system calls; (C) is false because a system call executes in kernel mode; (D) is false because `printf` is a library function, not a system call.

Q2 (B). The trap carries only a number. The kernel uses it as a direct index into the system-call table, whose entry is a function pointer to the handler — a lookup, not a name search. The privilege level is changed by the trap before dispatch, not selected by the number, so (D) is false.

Q3 (B). A compiled program contains the system-call *number*, not the service name, so the numbering is part of the application binary interface. Renumbering would make every existing binary request the wrong handler. New calls are therefore appended and old numbers are preserved. The trap number width (C) and the argument convention (D) are unrelated constraints.

Q4 (B). Register passing is bounded by the number of argument registers: six values fit, nine do not, and they cannot be "packed" into six registers without losing information. The block method stores all nine arguments contiguously and passes one address; the kernel copies the block and validates every field (including that the whole block lies in user memory). The stack is also viable in principle, but (C) is false because the kernel must still read and copy the arguments from the user stack.

Q5 (B). Every argument is untrusted input. The kernel checks that the range lies in the calling process's address space and, on failure, returns `EFAULT` through the normal return path; it never dereferences the pointer. Dereferencing (A) or copying the data (D) would let user code read kernel memory, defeating the privilege boundary. An immediate kill (C) is not how a bad user pointer is reported.

Q6 40. The first six arguments travel in registers and are not part of the memory block. The block holds $11 - 6 = 5$ arguments at 8 bytes each: $5 \times 8 = 40$ bytes. The kernel copies those 40 bytes into kernel memory and validates the block's address and extent before reading any field.

Q7 32. The file is 96 KiB and the buffer is 8 KiB, so the number of full buffers is $96/8 = 12$. Thus `cp` issues 12 `read` and 12 `write` calls. Adding the remaining named calls:

1 (`fork`) + 1 (`exec`) + 2 (`open`) + 12 (`read`) + 12 (`write`) + 2 (`close`) + 1 (`exit`) + 1 (`wait`) = 32.

Q8 **256.** Version A copies 1 MiB = 1024 KiB in 4 KiB buffers, so it performs $1024/4 = 256$ iterations. Each iteration issues one `read` and one `write`, giving 512 user-to-kernel transitions. Version B issues one `read` and one `write`, giving 2 transitions. The factor is $512/2 = 256$. The bytes moved are identical; only the number of boundary crossings changes.

Q9 **3.** File management covers `open`, `lseek`, and `close`. The other seven are process control (`fork`, `wait`); device management (`ioctl`); information maintenance (`getpid`); communication (`socket`, `send`); and protection (`chmod`).

Q10 **351.** Appending twelve calls extends the table from 340 to $340 + 12 = 352$ entries, numbered 0 through 351. Withdrawing five calls from service does not free their numbers, because those numbers remain part of the ABI. Hence the largest valid number is $352 - 1 = 351$.

Concepts That Click